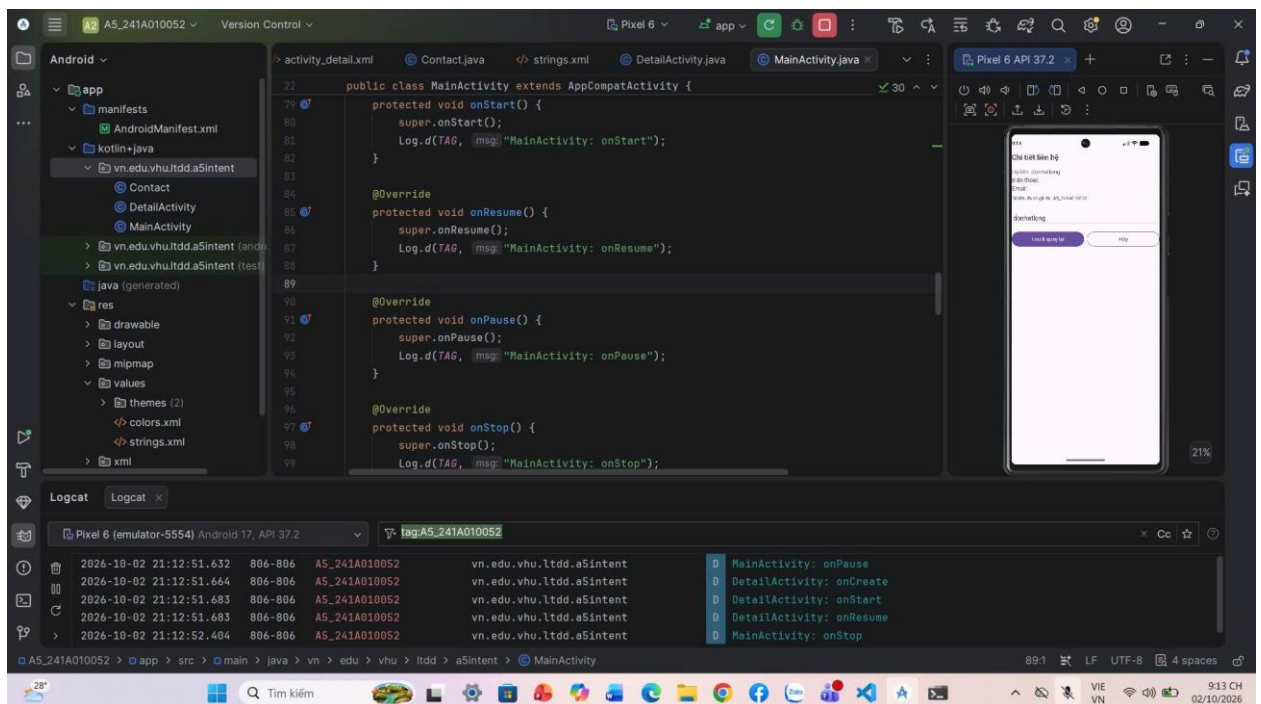
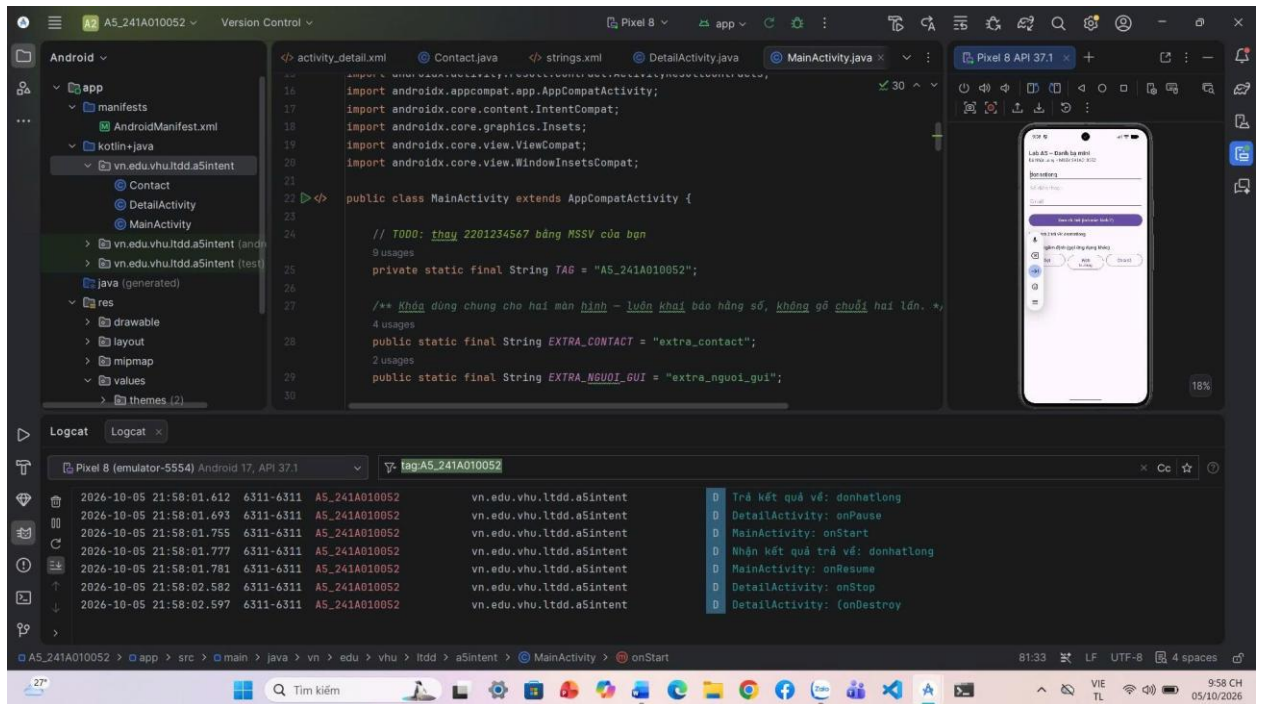


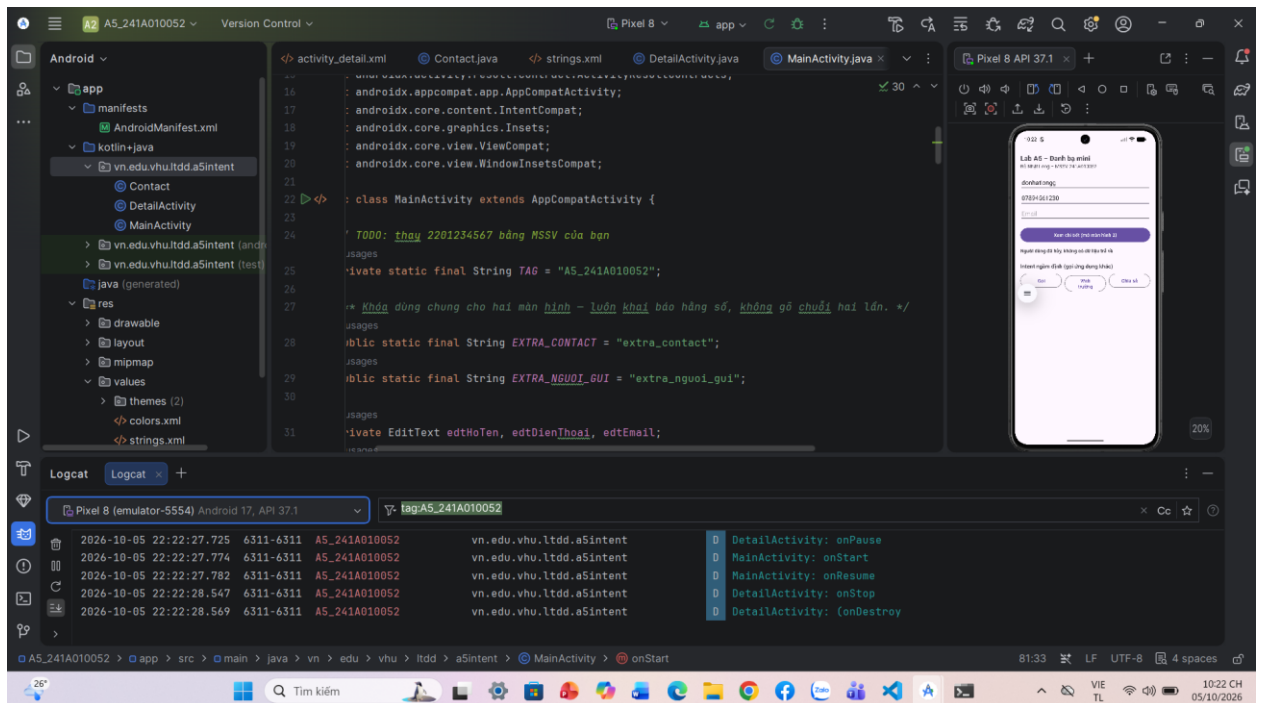
#	Thao tác	Quan sát (ghi vào báo cáo)
1	Bấm "Xem chi tiết"	Khi mở DetailActivity từ MainActivity, Logcat ghi nhận MainActivity.onPause(), tiếp tục DetailActivity.onCreate(), onStart(), onResume(), rồi mới đến MainActivity.onStop(). Điều này cho thấy MainActivity chỉ dừng hoàn toàn khi DetailActivity đã hiển thị. Không có MainActivity.onDestroy(), nên MainActivity chưa bị hủy.
2	Ở màn hình 2 bấm "Lưu & quay lại"	Bấm lưu và quay lại DetailActivity trả kết quả, dữ liệu donhatlong được gửi về tại MainActivity. MainActivity gọi onStart() rồi nhận kết quả trước khi gọi onResume(). Sau đó, DetailActivity gọi onStop() và onDestroy(). Điều này cho thấy dữ liệu đã được trả về khi DetailActivity kết thúc.
3	Ở màn hình 2 nhấn Back	Khi quay lại MainActivity, Logcat ghi nhận DetailActivity.onPause(), sau đó MainActivity.onStart() và onResume(). Tiếp theo, DetailActivity gọi onStop() rồi onDestroy(). Điều này cho thấy MainActivity được đưa trở lại trạng thái hoạt động trước khi DetailActivity bị hủy.
4	Xoay màn hình khi đang ở màn hình 2	Khi xoay màn hình, DetailActivity lần lượt gọi onPause(), onStop() và onDestroy(). Sau đó, một DetailActivity mới được tạo và gọi onCreate(), onStart() và onResume(). Điều này cho thấy khi cấu hình màn hình thay đổi, Activity bị hủy và tạo lại.
5	Bấm "Chia sẻ" rồi quay lại app	Khi bấm Chia sẻ, MainActivity gọi onPause(), onStop(). Khi quay lại ứng dụng, Activity gọi onStart(), onResume(). Không có onDestroy() nên MainActivity không bị hủy chỉ tạm dừng khi giao diện chia sẻ đang mở.



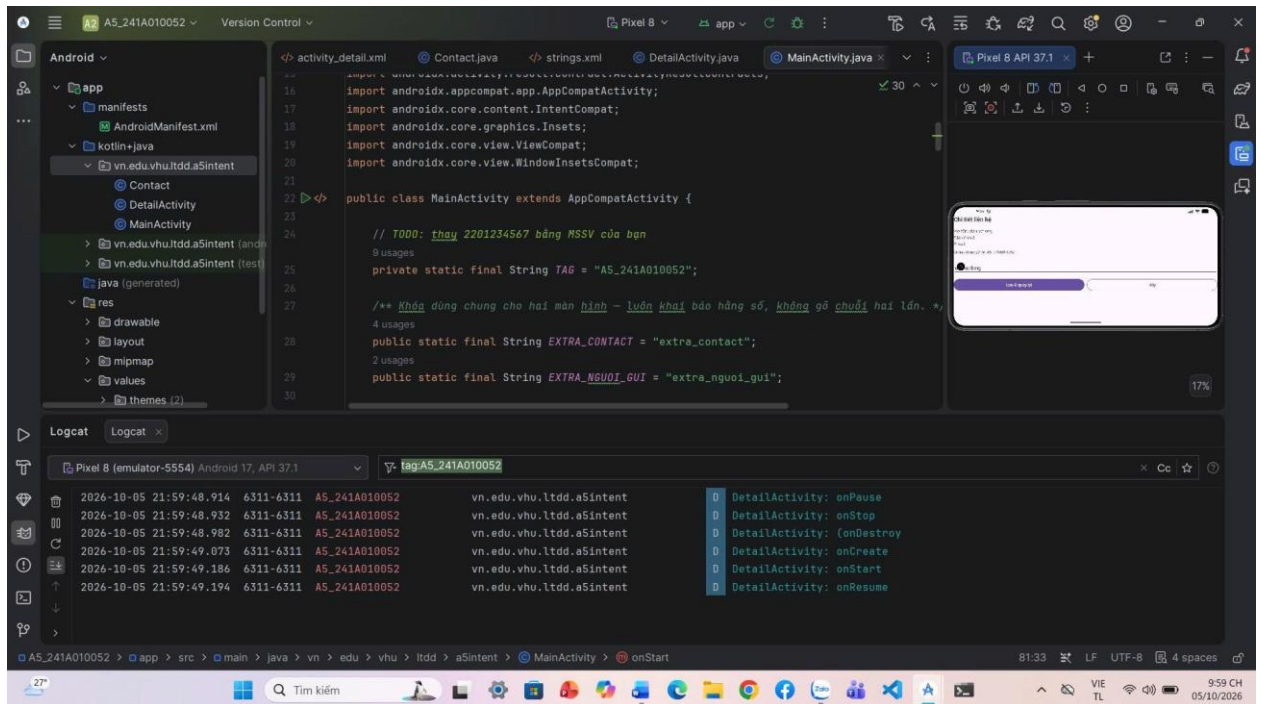
Thí nghiệm 1



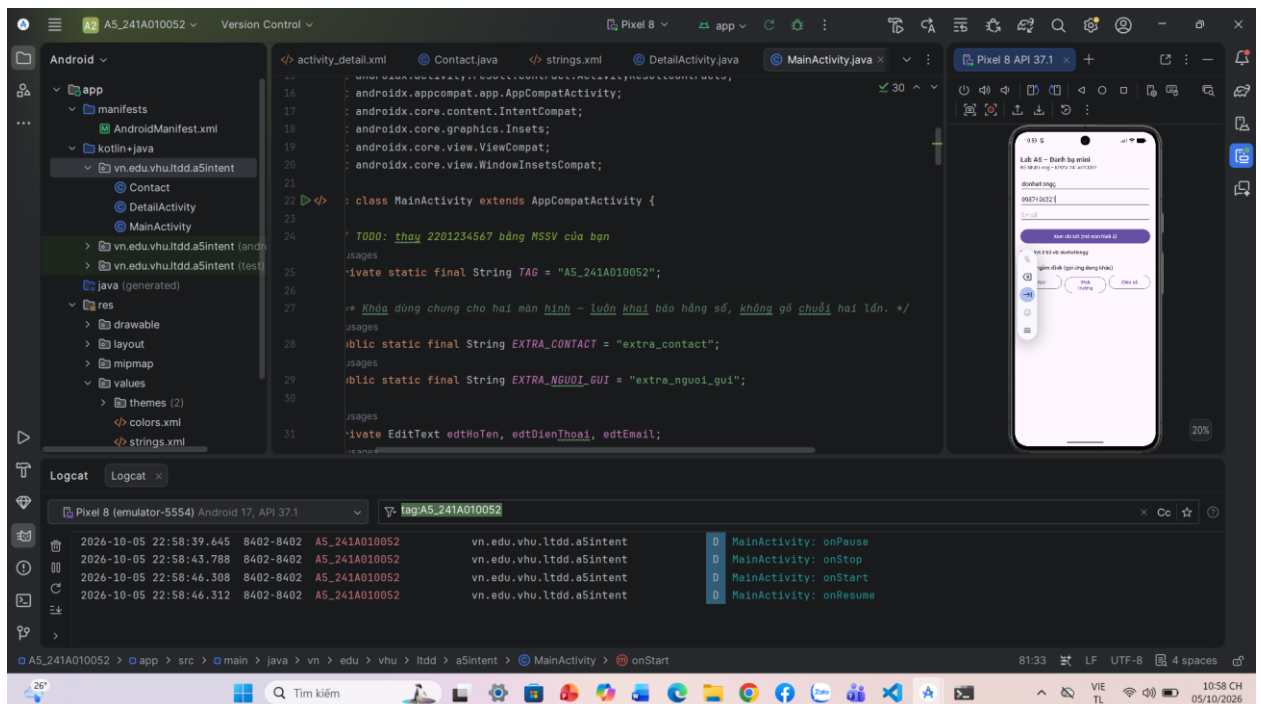
Thí nghiệm 2



Thí nghiệm 3



Thí nghiệm 4



Thí nghiệm 5

Trả lời câu hỏi thí nghiệm vòng đời hai màn hình:

Câu 1: Ở kịch bản 1, vì sao A.onStop chạy sau B.onResume chứ không phải trước?

onPause() được gọi khi A mất trạng thái foreground; còn onStop() được gọi khi A không còn hiển thị. Đây là hai mốc vòng đời khác nhau. Android cần đưa B vào trạng thái hoạt động ngay sau khi A tạm dừng, nhưng không yêu cầu phải hoàn tất onStop() của A trước khi gọi onResume() của B.

Câu 2: Ở kịch bản 3, resultCode nhận được là gì? Dữ liệu có được trả về không? Vì sao?

Ở kịch bản 3, khi DetailActivity đóng mà không gọi setResult(), MainActivity nhận RESULT_CANCELED (0). Không có dữ liệu được trả về vì Activity kết thúc mà không đặt kết quả kèm dữ liệu.

Câu 3: Ở kịch bản 4, dữ liệu nhận qua Intent có còn không? Giải thích (gợi ý: getIntent() trả về gì sau khi Activity được tạo lại?).

Có. Khi Activity được tạo lại, getIntent() vẫn trả về Intent đã dùng để khởi chạy Activity, bao gồm các extras ban đầu. Vì vậy, dữ liệu nhận qua Intent vẫn còn, trừ khi ứng dụng chủ động thay đổi hoặc xóa dữ liệu đó.

Câu 4: Ở kịch bản 4, nếu bạn đã sửa họ tên trong ô nhập mà chưa bấm Lưu thì nội dung đó có còn không? Muốn giữ thì phải làm gì (nhắc lại Lab A2)?

Nếu Activity được tạo lại, nội dung trong EditText thường được Android tự khôi phục khi ô nhập có id. Nhưng thay đổi đó chưa được lưu thành dữ liệu lâu dài. Để chủ động giữ nội dung theo cách đã học ở Lab A2, lưu giá trị trong onSaveInstanceState().

Trả lời câu hỏi ôn tập

1. Khác nhau giữa Intent tường minh và ngầm định? Cho một ví dụ mỗi loại.

Intent tường minh: Chỉ rõ thành phần cần mở, thường dùng để chuyển giữa các Activity trong cùng ứng dụng

Intent ngầm định: Chỉ mô tả hành động cần thực hiện, không chỉ rõ thành phần; Android chọn ứng dụng phù hợp để xử lý.

2. Vì sao phải dùng hằng số cho khóa của extras?

Vì hằng số cho khóa của extras giúp tránh lỗi do gõ sai hoặc dùng tên khóa không nhất quán giữa nơi gửi và nơi nhận dữ liệu. Việc này cũng giúp mã dễ bảo trì hơn, vì nếu cần đổi khóa thì chỉ cần sửa ở một nơi.

3. Nêu các bước để nhận dữ liệu trả về từ màn hình thứ hai (cả hai phía).

Phía MainActivity (màn hình gọi):

Mở `DetailActivity` bằng `startActivityForResult()` và truyền mã yêu cầu.

Ghi đè `onActivityResult()`, kiểm tra `requestCode` và `resultCode`.

Nếu kết quả hợp lệ (`RESULT_OK`), lấy dữ liệu từ `Intent` trả về bằng `get...Extra()`.

Phía `DetailActivity` (màn hình trả kết quả):

Tạo `Intent` chứa dữ liệu cần gửi bằng `putExtra()`.

Gọi `setResult(RESULT_OK, intent)` để đặt kết quả.

Gọi `finish()` để đóng màn hình và trả kết quả về `MainActivity`.

4. Khi mở màn hình 2, màn hình 1 đi qua những callback nào? Màn hình 1 có bị hủy không?

Khi mở màn hình 2, màn hình 1 đi qua `onPause()` và `onStop()`, khi không còn hiển thị màn hình 1 không bị hủy chỉ vì mở màn hình 2, nó vẫn nằm trong back stack và có thể tiếp tục khi quay lại.

5. Vì sao nên dùng try-catch thay cho `resolveActivity()` khi mở Intent ngầm định?

Vì try-catch để xử lý trực tiếp trường hợp không có ứng dụng phù hợp mở `Intent`: nếu không mở được, ứng dụng bắt `ActivityNotFoundException` thay vì bị lỗi.